

# Luisa's Project Documentation: COTB Python Game Notes

Luisa Narvaz Blankenburg  
Orientação: Prof. Andre Luiz de Moraes

2025/2026

## Sumário

|           |                                                                |          |
|-----------|----------------------------------------------------------------|----------|
| <b>1</b>  | <b>Checklist de Atividades</b>                                 | <b>2</b> |
| <b>2</b>  | <b>Introduction</b>                                            | <b>3</b> |
| <b>3</b>  | <b>Background</b>                                              | <b>3</b> |
| <b>4</b>  | <b>Related Works</b>                                           | <b>3</b> |
| <b>5</b>  | <b>Experiment Design</b>                                       | <b>3</b> |
| <b>6</b>  | <b>Results</b>                                                 | <b>4</b> |
| <b>7</b>  | <b>Discussion</b>                                              | <b>4</b> |
| <b>8</b>  | <b>Final considerations and Future Works</b>                   | <b>4</b> |
| <b>9</b>  | <b>Conclusion</b>                                              | <b>4</b> |
| <b>10</b> | <b>Links importantes</b>                                       | <b>4</b> |
| 10.1      | Luisa's repository Files (Google Drive, Architecture design... | 4        |
| 10.2      | Instalação e configuração Raspberry                            | 4        |
| 10.3      | Protótipo interface visual                                     | 4        |
| 10.4      | Criando som com Sonic Pi                                       | 4        |

### Próximas Atividades

- **Tutorial 9:** Implementação de Score, Estímulo-Resposta e Feedback (Seção 10.4).
  - Registrar vídeo da etapa para comparação futura.

- **Tutorial 10:** Interação com botões e GPIO (Seção 10.4).
- **Escrita Científica:** Redigir capítulos de Background e Related Works.

## 1 Checklist de Atividades

| Etapa               | Tarefa                              | Observação     | Status                              | Previsão |
|---------------------|-------------------------------------|----------------|-------------------------------------|----------|
| Etapa 0:            | Criar referencial teórico.          |                | <input checked="" type="checkbox"/> | Ago-3    |
|                     | Estruturar repositório.             |                | <input checked="" type="checkbox"/> | Ago-3    |
|                     | Planejamento inicial.               |                | <input checked="" type="checkbox"/> | Ago-4    |
|                     | Refinar ideia central.              |                | <input checked="" type="checkbox"/> | Ago-4    |
|                     | Público-alvo (7-10 anos).           |                | <input checked="" type="checkbox"/> | Ago-4    |
|                     | Tipo de tarefa.                     | Altura/Melodia | <input checked="" type="checkbox"/> | Ago-4    |
|                     | Verificar acesso ao público-alvo.   |                | <input type="checkbox"/>            | Set-1    |
|                     | Plataforma Web/Desktop.             |                | <input checked="" type="checkbox"/> | Set-1    |
|                     | Configurar UTM/VM.                  |                | <input checked="" type="checkbox"/> | Set-1    |
|                     | Interações gráficas.                |                | <input checked="" type="checkbox"/> | Set-2    |
|                     | Uso do terminal e script Python.    |                | <input checked="" type="checkbox"/> | Set-3    |
|                     | Lógica e regras do jogo.            |                | <input checked="" type="checkbox"/> | Set-3    |
|                     | Protótipo visual.                   |                | <input checked="" type="checkbox"/> | Set-3    |
|                     | Estímulos musicais.                 |                | <input type="checkbox"/>            | Set-4    |
|                     | Mapear sons a eventos.              |                | <input type="checkbox"/>            | Set-4    |
|                     | Fluxograma de interação.            |                | <input checked="" type="checkbox"/> | Set-4    |
|                     | Protótipo final de interface.       |                | <input checked="" type="checkbox"/> | Set-4    |
|                     | Instalar ambiente Raspbian.         | UTM/VirtualBox | <input checked="" type="checkbox"/> | Ago-4    |
|                     | Implementar script de som (pygame). |                | <input checked="" type="checkbox"/> | Out-1    |
|                     | Reprodução de áudio.                |                | <input checked="" type="checkbox"/> | Out-1    |
|                     | Testar teclado → som.               |                | <input checked="" type="checkbox"/> | Out-2    |
|                     | Protótipo funcional integrado.      |                | <input checked="" type="checkbox"/> | Out-2    |
| Etapa 3B:           | Tela inicial (Nome/Escola).         |                | <input checked="" type="checkbox"/> | Out-3    |
|                     | Progressão de fases.                |                | <input type="checkbox"/>            | Out-4    |
|                     | Feedback sonoro integrado.          |                | <input type="checkbox"/>            | Out-4    |
| Etapa 5: Coleta de  | Sessões com participantes.          |                | <input type="checkbox"/>            | Nov-1    |
|                     | Organizar dados.                    |                | <input type="checkbox"/>            | Nov-2    |
|                     | Análises estatísticas.              |                | <input type="checkbox"/>            | Nov-3    |
|                     | Gráficos comparativos.              |                | <input type="checkbox"/>            | Nov-3    |
| Etapa 6: Redação do | Template Overleaf.                  |                | <input checked="" type="checkbox"/> | Nov-2    |
|                     | Redação (Metodologia/-Resultados).  |                | <input type="checkbox"/>            | Nov-4    |

|  |                           |  |                          |       |
|--|---------------------------|--|--------------------------|-------|
|  | Submeter ao COTB<br>2026. |  | <input type="checkbox"/> | Nov-4 |
|--|---------------------------|--|--------------------------|-------|

## 2 Introduction

- Breve Explicação sobre TDAH e suas características
- Justificativa de uso da Faixa etária
- Problemas que o TDAH acarreta no dia a dia (exemplificações mais concretas)
- Recursos e ferramentas que são adotadas para auxiliar crianças ou pessoas com este déficit
- Trazer a proposta do trabalho com a possível solução ou método que será implementado
- Descrever as seções do trabalho juntamente com o que se trata

## 3 Background

- TDAH e suas características
- Microcontroladores inteligentes [Raspberry, Arduino]
- Parte física e biológica [sentidos dos seres humanos]
- Sons e suas frequências [como os sons afetam ou mudam as percepções]
- Tecnologias para Estímulo Multisensorial

## 4 Related Works

## 5 Experiment Design

Aqui é onde detalharemos como é montada a estrutura da implementação de infraestrutura (se foram recursos de virtualização como virtualbox ou docker) e em termos de código (uso do pip para instalação das bibliotecas). Depois detalhando a organização geral, linguagem adotada, recursos empregados em termos de bibliotecas (sons, raspberry, video, etc... ).

## **6 Results**

## **7 Discussion**

## **8 Final considerations and Future Works**

## **9 Conclusion**

## **10 Links importantes**

### **10.1 Luisa's repository Files (Google Drive, Architecture design...**

- Anotações de Luisa no Google Drive
- Link para o Draw.io da arquitetura
- Link Paper COTB
- Link do GitHub

### **10.2 Instalação e configuração Raspberry**

- Playlist de curso de Extensão de prof. André no YouTube

### **10.3 Protótipo interface visual**

- Interface no Canva

### **10.4 Criando som com Sonic Pi**

- Introdução ao Sonic Pi
- Criando música com Sonic Pi

## Tutorial 1: Primeiros Passos com UTM e Raspberry Pi OS

Nesta etapa, o objetivo é interagir com o ambiente Raspberry Pi via **UTM**, facilitando a emulação gráfica inicial.

### Objetivos

- Familiarizar-se com a interface gráfica do Raspberry Pi OS.
- Compreender o conceito de máquinas virtuais e emulação.

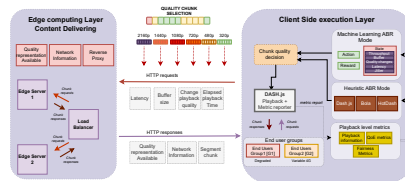


Figura 1: Arquitetura Geral do Experimento.

### Resultados de Aprendizagem

- Entender a virtualização do sistema e navegação básica.

## Tutorial 2: Uso do Terminal no Raspberry Pi OS via UTM

Introdução ao terminal Linux: comandos básicos, instalação de pacotes e execução de scripts Python.

### Objetivos

- Abrir e utilizar o terminal do Raspberry Pi OS.
- Instalar pacotes com `apt` e executar scripts Python.

```
sudo apt-get update && sudo apt-get upgrade  
sudo apt-get install python3 python3-pip -y  
python3 ola.py
```

### Resultados de Aprendizagem

- Autonomia no gerenciamento de arquivos e execução de software via CLI.

## Tutorial 3: Design do Jogo e Feedback Musical

```
pygame.mixer.init()
screen = pygame.display.set_mode((400, 300))
pygame.display.set_caption("Teste de Sons")

som_acerto = pygame.mixer.Sound("acerto.wav")
som_erro = pygame.mixer.Sound("erro.wav")

rodando = True
while rodando:
    for evento in pygame.event.get():
        if evento.type == pygame.QUIT:
            rodando = False
        if evento.type == pygame.KEYDOWN:
            if evento.key == pygame.K_a:
                som_acerto.play()
            elif evento.key == pygame.K_e:
                som_erro.play()

pygame.quit()
```

### Primeiros Exercícios Práticos

- Criar um script que toque diferentes sons com base em teclas pressionadas.
- Trocar os sons por opções escolhidas no tutorial anterior.
- Criar variações para testar sons diferentes em sequência.

### Resultados de Aprendizagem Esperados

- Compreender como integrar eventos do teclado com reprodução de sons em Python.
- Criar um programa básico, mas funcional, com estrutura de loops, condicionais e eventos.
- Preparar terreno para expansão do código em forma de jogo ou aplicação educacional.

## Tutorial 5: Telas e Gráficos com Pygame

Nesta etapa inicia-se a implementação em Python utilizando a biblioteca `pygame`. O objetivo é criar a janela principal, o laço de execução e um estímulo visual (estrela piscando).

### Guia de Aprendizagem

1. Criar a pasta do projeto e o ambiente virtual:

```
mkdir -p ~/projetos/musicalizando && cd ~/projetos/musicalizando
python3 -m venv .venv
source .venv/bin/activate
pip install pygame
```

2. Código base para a estrela piscando:

```
import pygame, sys
WIDTH, HEIGHT = 800, 480
FPS = 60

def main():
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    clock = pygame.time.Clock()
    blink_timer = 0.0
    star_visible = True

    while True:
        dt = clock.tick(FPS) / 1000.0
        for event in pygame.event.get():
            if event.type == pygame.QUIT: pygame.quit(); sys.exit()

        blink_timer += dt
        if blink_timer >= 0.8:
            star_visible = not star_visible; blink_timer = 0.0

        screen.fill((20, 30, 80))
        if star_visible:
            pygame.draw.circle(screen, (255, 215, 0), (WIDTH//2,
                ↪ HEIGHT//2), 30)
            pygame.display.flip()

if __name__ == "__main__": main()
```

### Exercícios Práticos

- Alterar o tamanho e a velocidade do piscar da estrela.
- Posicionar duas estrelas em ritmos diferentes.

## Tutorial 6: Score e Feedback de Estímulo-Resposta

Mapeamento de entrada do usuário (teclado/clique) e associação com feedback positivo/-negativo.

### Guia de Aprendizagem

1. Testar o áudio no Raspberry Pi:

```
import pygame
pygame.mixer.init()
s = pygame.mixer.Sound("acerto.wav")
s.play()
```

2. Mapear tecla 'A' para acerto:

```
if event.type == pygame.KEYDOWN and event.key == pygame.K_a:
    if star_visible:
        pontos += 1; acerto_snd.play()
    else:
        pontos = max(0, pontos-1); erro_snd.play()
```

### Exercícios Práticos

- Adicionar feedback visual (ex: borda verde para acerto, vermelha para erro).
- Exibir a pontuação atualizada na tela usando `pygame.font`.

## Tutorial 7: Fases e Progressão de Sequências

Nesta etapa organizamos a lógica de fases, adicionamos sequências (listas de estímulos) e tempo-limite de resposta. Ao completar a sequência, a fase é concluída e pode-se avançar.

### Objetivos

- Definir fases com parâmetros (nº de estímulos, tempo de resposta, velocidade).
- Implementar sequência de alvos e avaliação de acerto dentro do tempo.
- Exibir barra de progresso ou contador de acertos da fase.

### Requisitos

- Concluir as Etapas 5 e 6.

### Guia de Aprendizagem

1. Criar um arquivo de configuração simples (`levels.py`) com as fases:

```
# levels.py
LEVELS = [
    {"name": "Fase 1", "seq_len": 4, "blink_period": 0.9, "timeout":
    ↪ 2.8},
    {"name": "Fase 2", "seq_len": 6, "blink_period": 0.7, "timeout":
    ↪ 2.0},
    {"name": "Fase 3", "seq_len": 8, "blink_period": 0.55, "timeout":
    ↪ 1.5},
]
```

2. Atualizar `main.py` para controlar sequência, tempo e progresso:

```

import pygame, sys, random, time
from levels import LEVELS

WIDTH, HEIGHT = 800, 480
FPS = 60
SKY = (20, 30, 80)
WHITE = (245, 245, 245)
GOLD = (255, 215, 0)
GREEN = (60, 180, 75)
RED = (200, 60, 60)

def desenhar_estrela(surface, x, y, size, color):
    # Desenha um polígono estelar de 5 pontas
    pts = [(x, y - size), (x + size*0.3, y - size*0.3), (x + size, y -
        ↪ size*0.3),
            (x + size*0.45, y + size*0.1), (x + size*0.6, y + size),
            (x, y + size*0.45), (x - size*0.6, y + size),
            (x - size*0.45, y + size*0.1), (x - size, y - size*0.3),
            (x - size*0.3, y - size*0.3)]
    pygame.draw.polygon(surface, color, pts)

def main():
    pygame.init(); pygame.mixer.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Musicalizando no Céu - Fases")
    clock = pygame.time.Clock()
    font = pygame.font.SysFont(None, 26)

    # Carregar sons (Certifique-se de que os arquivos existem em
    ↪ assets/sounds/)
    # acerto_snd = pygame.mixer.Sound("assets/sounds/acerto.wav")
    # erro_snd = pygame.mixer.Sound("assets/sounds/erro.wav")

    level_idx = 0; pontos = 0

    while level_idx < len(LEVELS):
        level = LEVELS[level_idx]
        seq_len = level["seq_len"]; blink_per = level["blink_period"];
        ↪ timeout_s = level["timeout"]
        sequence = [random.choice([0,1]) for _ in range(seq_len)]
        seq_pos = 0; star_visible = False; blink_timer = 0.0; stim_timer
        ↪ = 0.0
        feedback_color = None; feedback_timer = 0.0; responded = False

        running_phase = True
        while running_phase:
            dt = clock.tick(FPS)/1000.0
            for event in pygame.event.get():
                if event.type == pygame.QUIT: pygame.quit(); sys.exit()
                if event.type == pygame.KEYDOWN and event.key ==
                ↪ pygame.K_a:
                    if not responded:
                        responded = True
                    if star_visible:
                        pontos += 1; feedback_color = GREEN
                    else:
                        pontos = max(0, pontos-1); feedback_color =
                        ↪ RED
                        feedback_timer = 0.3
                        11

            # Lógica de piscar e sequência
            blink_timer += dt
            if blink_timer >= blink_per:

```

### **Exercícios Práticos**

- Trocar a regra da sequência para notas (ex.: 3 sons diferentes).
- Ajustar `seq_len` e `timeout` no arquivo `levels.py`.
- Exibir uma tela de “Fase Concluída” com o score final.

### **Resultados de Aprendizagem Esperados**

- Controle de fluxo por fases e temporização por estímulo.
- Implementação de progressão e finalização de fase.

## Tutorial 8: Tela Inicial e Registro do Jogador

Esta etapa implementa a tela inicial minimalista: campos para **Nome** e **Escola**.

### Objetivos

- Exibir tela inicial com campos de texto e botão "Começar".
- Validar entradas antes de avançar.

```
while mode == "menu":  
    # ... lógica de entrada de texto ...  
    if button_rect.collidepoint(event.pos):  
        if player_name.strip() and player_school.strip():  
            mode = "phase1"
```

### Resultados de Aprendizagem Esperados

- Entender a interação básica entre UI, eventos e estado do jogo.
- Ter um fluxo de entrada do jogador funcional que inicia a Fase 1.
- Preparar o terreno para implementar a lógica da Fase 1 no próximo tutorial.

## Tutorial 9: Fase 1 com Score e Feedback

Implementação da mecânica básica: quatro alvos (setas), um alvo destacado por rodada e atualização do placar.

### Objetivos

- Implementar a lógica de estímulo-resposta.
- Exibir HUD com Nome, Escola e Pontuação.

```
if event.key in STAR_KEYS:
    if event.key == target_key:
        score += 1
        feedback = "ACERTOU"
```

### Resultados de Aprendizagem

- Interação dinâmica e feedback imediato em tempo real.

## Tutorial 10: Interação com Botões (GPIO)

Integração de botões físicos via **GPIO** e alternativas de simulação via software para desenvolvimento em ambientes sem hardware.

### Dicas Técnicas

- Sempre use apenas 3.3V nos GPIOs (não use 5V).
- Botões simples devem ser conectados entre o pino GPIO e GND, com pull-up interno habilitado.

### Guia de Aprendizagem

1. Exemplo de botão físico com `gpiozero`:

```
from gpiozero import Button
def on_hw_button():
    print("Botão físico pressionado!")

btn = Button(17, pull_up=True, bounce_time=0.05)
btn.when_pressed = on_hw_button
```

2. Exemplo de Mock GPIO (para testes locais em PC):

```
from gpiozero import Device, Button
from gpiozero.pins.mock import MockFactory
Device.pin_factory = MockFactory()
btn = Button(17, pull_up=True)

# Simular pressão via código
pin = Device.pin_factory.pin(17)
pin.drive_low(); pin.drive_high()
```

### Exercícios Práticos

- Adicionar um segundo botão para “Pausar” o jogo.
- Registrar timestamps de pressões em CSV (tempo de reação).

## Tutorial 11: Layout Dinâmico e Responsivo

Nesta etapa aprendemos a posicionar elementos de forma relativa ao tamanho da tela, permitindo que o jogo rode em diferentes resoluções.

### Guia de Aprendizagem

1. Criar uma função auxiliar para calcular posições relativas:

```
def pos_rel(x_frac, y_frac, width, height):  
    """ Retorna posição relativa dado percentual da tela """  
    return int(width * x_frac), int(height * y_frac)
```

2. Criar botões com base em porcentagens:

```
WIDTH, HEIGHT = 900, 600  
start_btn_rect = pygame.Rect(0, 0, WIDTH//4, HEIGHT//12)  
start_btn_rect.center = pos_rel(0.5, 0.7, WIDTH, HEIGHT)
```

### Exercícios Práticos

- Criar uma função `resize_ui()` para reposicionamento dinâmico.
- Testar o jogo em diferentes resoluções (fullscreen, 1280x720).

## Tutorial 12: Trilha Sonora e Ambiência

Nesta etapa, Luisa aprenderá a incorporar sons ao jogo, criando músicas de fundo e efeitos específicos para eventos de vitória e derrota.

### Checklist de Atividades

Escolher trilha, som de vitória e som de derrota (BeepBox/Mixkit).

Salvar arquivos em `.mp3` ou `.wav` na pasta `./sounds/`.

Inserir código Python para reprodução sonora.

### Sites Recomendados

- [Freesound.org](https://freesound.org) Efeitos e sons gratuitos [*prontos para baixar*].
- [BeepBox.co](https://beepbox.co) Cria melodias curtas em estilo 8-bit [*para criação própria*].
- [Soundtrap.com](https://soundtrap.com) Editor online para trilhas elaboradas.

```
pygame.mixer.music.load("sounds/trilha_fase.mp3")  
pygame.mixer.music.play(-1) # Loop infinito
```



Figura 2: Interface do Soundtrap: ferramenta para criação de trilhas sonoras complexas.

### Feedback e Gamificação

A utilização estratégica de áudio e visual (fanfarras, ícones de vitória) reforça o engajamento e o aprendizado atencional.

### Resultados de Aprendizagem

- Uso estratégico de áudio para reforçar o engajamento e feedback.

### Conclusão

Com esta etapa, o jogo passa a ter um componente sonoro dinâmico que reforça o envolvimento da jogadora.

## Tutorial 13: Introdução ao BeepBox e Tema Mario

Apresentação do **BeepBox** para criação de trilhas 8-bit e recriação do tema do *Super Mario Bros.* em versão simplificada.

### Guia Musical (Mario Theme)

E E - E - C E - G - - - G  
C - G - E - A B A A - -

### Resultados de Aprendizagem

- Compreensão de melodia, ritmo e exportação de áudio para jogos.



Figura 3: Interface do BeepBox para criação de trilhas 8-bit.

## Tutorial: Fluxo de Trabalho Git

Guia para versionamento colaborativo utilizando Git e GitHub.

### Objetivos

- Sincronizar o código entre diferentes máquinas.

- Manter o histórico de versões seguro.

```
git add .  
git commit -m "Mensagem clara"  
git push origin main  
  
# 3. Ver o que mudou  
git status  
  
# 4. Adicionar as alterações  
git add .  
  
# 5. Criar o commit  
git commit -m "Mensagem clara"  
  
# 6. Enviar para o GitHub  
git push origin main
```

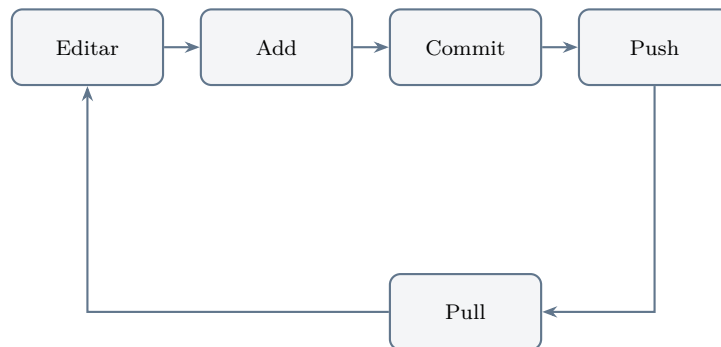


Figura 4: Ciclo de desenvolvimento com Git.

## Conclusão

Seguindo este fluxo, cada colaborador pode trabalhar de forma independente e sincronizada.

## Tutorial 14: SSH e Cabo RJ45

Estabelecimento de conexão direta entre Raspberry Pi e Computador para configuração inicial.

### Objetivos

- Acessar o terminal via SSH.
- Ativar o serviço sem monitor/teclado.

```
ssh pi@raspberrypi.local
```

### Dica de Ouro

*Crie um arquivo vazio chamado **ssh** na partição boot do SD para ativar o acesso automático.*

## Tutorial 15: Acesso Gráfico via VNC

Acesso ao desktop do Raspberry Pi para facilitar o desenvolvimento visual.

### Objetivos

- Ativar VNC Server via `raspi-config`.
- Conectar usando VNC Viewer.

```
sudo raspi-config # Interface Options -> VNC -> Enable
```

## Checklist de Revisão Estrutural

Esta seção contém os modelos de reescrita e ajustes formais necessários para a padronização acadêmica do documento.

### 1. Abstract

- ☒ Substituir expressões fortes como “comprovada a eficácia” por termos como “avaliada a viabilidade”.

#### Modelo de Reescrita:

*“para que possa ser avaliada a viabilidade e o impacto preliminar da ferramenta proposta.”*

- ☒ Reformular “potencial pedagógico” para linguagem não conclusiva.

#### Modelo de Reescrita:

*“Espera-se que a ferramenta apresente indícios de viabilidade pedagógica em contexto educacional.”*

- ☒ Inserir frase explícita indicando que a proposta não substitui tratamento clínico ou farmacológico.

#### Modelo de Inserção:

*“Ressalta-se que a proposta não substitui intervenções clínicas ou farmacológicas, atuando como ferramenta complementar de apoio educacional.”*

- ☒ Tornar mais claro o delineamento metodológico previsto.

#### Modelo de Inserção:

*“O estudo prevê delineamento quase-experimental com comparação entre condições com e sem estímulo musical estruturado.”*

### 2. Introdução

- ☒ Reescrever o parágrafo que inicia com “Diferentemente da abordagem medicinal tradicional...” evitando comparação direta com medicina tradicional.

#### Modelo de Reescrita:

*“Além das intervenções farmacológicas amplamente utilizadas, pesquisas recentes indicam que abordagens complementares baseadas em estímulos cognitivos e musicais podem contribuir para o desenvolvimento da atenção em crianças com TDAH.”*

- ☒ Inserir hipótese formal da pesquisa.

#### Modelo de Inserção:

*“A hipótese central deste estudo é que crianças expostas a estímulos musicais estruturados durante atividades interativas apresentarão melhor desempenho em tarefas de atenção sustentada quando comparadas à condição sem estímulo musical estruturado.”*

- ☒ Definir variáveis independentes (tipo de estímulo musical).

#### Modelo de Inserção:

*“A variável independente corresponde ao tipo de estímulo musical aplicado durante a atividade.”*

- ☒ Definir variáveis dependentes (tempo de resposta, número de acertos, evolução por fase).

**Modelo de Inserção:**

*“As variáveis dependentes incluem tempo de resposta, taxa de acertos e evolução do desempenho ao longo das fases do jogo.”*

- ☒ Reforçar explicitamente que a proposta é complementar ao tratamento clínico.

**Modelo de Inserção:**

*“A proposta apresentada possui caráter complementar, não substituindo intervenções clínicas ou terapêuticas previamente estabelecidas.”*

### 3. Fundamentação Teórica

- ☒ Substituir termos como “portador do transtorno” por “criança com TDAH”.

**Modelo de Substituição:**

*Trocar todas as ocorrências de “portador do transtorno” por “criança com TDAH”.*

- ☒ Inserir referências recentes (2022–2025).

**Modelo de Inserção:**

*Inserir estudos recentes que abordem música e atenção, intervenções não farmacológicas no TDAH e jogos digitais cognitivos.*

- ☒ Remover a Tabela comparativa Arduino vs Raspberry Pi.

- ☒ Substituir a tabela por um parágrafo técnico objetivo justificando a escolha do Raspberry Pi.

**Modelo de Inserção:**

*“A escolha do Raspberry Pi deve-se à sua maior capacidade de processamento, compatibilidade com bibliotecas multimídia como Pygame e possibilidade de registro automatizado de métricas atencionais em tempo real.”*

### 4. Trabalhos Relacionados

- ☐ Tornar a análise dos GAPS mais crítica e aprofundada.
- ☐ Inserir parágrafo final explicitando claramente a contribuição original do trabalho.

**Modelo de Inserção:**

*“Diferentemente dos trabalhos analisados, esta proposta integra estímulos musicais estruturados, coleta automatizada de métricas atencionais e aplicação em ambiente educacional, configurando uma abordagem interdisciplinar entre tecnologia acessível e intervenção cognitiva.”*

- ☐ Evitar linguagem superficial ou apenas descritiva.

### 5. Metodologia

- ☒ Especificar o tipo de estudo.

**Modelo de Inserção:**

*“O estudo caracteriza-se como quase-experimental, com delineamento comparativo entre duas condições experimentais.”*

☒ Inserir descrição formal do delineamento experimental.

☒ Considerar inclusão de grupo controle.

**Modelo de Inserção:**

*“Os participantes poderão ser divididos em Grupo Experimental (com estímulo musical estruturado) e Grupo Controle (sem estímulo musical adicional), permitindo comparação entre condições.”*

☐ Especificar testes estatísticos previstos.

**Modelo de Inserção:**

*“Os dados serão analisados por meio de teste  $t$  para comparação entre médias e ANOVA para análise entre fases, adotando-se nível de significância de  $p < 0,05$ .”*

☒ Substituir expressões especulativas como “será possível observar”.

**Modelo de Substituição:**

*Trocar por: “serão analisadas variações nos indicadores de desempenho atencional”.*

## 6. Design do Jogo

☐ Especificar parâmetros técnicos dos estímulos sonoros.

**Modelo de Inserção:**

*“Os estímulos sonoros serão configurados com frequência entre 400Hz e 800Hz, duração média de 300ms, ritmo entre 60 e 90 BPM e intervalo aproximado de 2 segundos entre estímulos.”*

## 7. Ferramentas Utilizadas

☐ Especificar modelo do Raspberry Pi.

**Modelo de Inserção:**

*“O sistema foi desenvolvido utilizando Raspberry Pi 4 Model B.”*

☒ Informar versão do Python e Pygame.

**Modelo de Inserção:**

*“Foram utilizadas as versões Python 3.11 e Pygame 2.x.”*

☒ Inserir link para repositório.

**Modelo de Inserção:**

*“O código-fonte será disponibilizado em repositório público para fins de reprodutibilidade científica.”*

## 8. Aspectos Éticos

☐ Criar subseção específica sobre Aspectos Éticos.

**Modelo de Texto:**

*“A participação ocorrerá mediante consentimento livre e esclarecido dos responsáveis legais e assentimento das crianças. Os dados serão anonimizados por meio de identificação alfanumérica, não sendo armazenadas informações pessoais sensíveis. O armazenamento seguirá as diretrizes da Lei Geral de Proteção de Dados (LGPD). Caso aplicável, o projeto será submetido a Comitê de Ética em Pesquisa.”*

## 9. Resultados

- ☒ Avaliar renomear seção para “Plano de Avaliação Experimental”.
- ☒ Substituir termos como “comprovam”.

### **Modelo de Substituição:**

*Trocar por: “indicam viabilidade técnica preliminar do sistema”.*

- ☒ Reduzir linguagem afirmativa sobre eficácia.

## 10. Considerações Finais

- ☒ Reformular linguagem conclusiva.

### **Modelo de Reescrita:**

*“Os resultados obtidos até o momento indicam viabilidade técnica do sistema, sendo necessária validação experimental com amostra ampliada para avaliação do impacto atencional.”*

- ☒ Destacar necessidade de validação futura.

## 11. Ajustes Formais

- ☒ Corrigir “Figure” para “Figura”.
- ☒ Corrigir “Table” para “Tabela”.
- ☐ Atualizar campos CCSXML.
- ☐ Revisão gramatical completa.